

Class Human

```
{  
    Int height,weight,maxspeed;
```

Public:

Void inputData()

```
{  
    Cout<<"enter weight";  
    Cin>>weight;  
    Cout<<"enter height";  
    Cin>>height;  
    Cout<<"enter maxspeed";  
    Cin>>maxspeed;  
}
```

Void run()

```
{  
    Cout << "He can run with maximum speed of"<<maxspeed;  
}  
};
```

```
Void main()
{
Human john,Nelson;
John.inputData();
John.run();
Nelson.inputData();
Nelson.run();
}
```

```
Class BankCustomer
{
    Int acct_number, balance;
```

```
Public:
```

```
Void inputData()
{
Cout<<"enter acct_number";
Cin>>acct_number;
Cout<<"enter balance";
```

```
Cin>>balance;  
}
```

```
Void output()  
{  
Cout << "AcctNumber is"<<acct_number;  
Cout << "Balance is"<<balance;  
  
}  
};
```

```
Void main()  
{  
BankCustomer Sridhar, Harshil;  
Sridhar.inputData();  
Sridhar.output();  
Harshil.inputData();  
Harshil.output();  
}
```

PYTHON

```
print("Hello, World!")
```

Python Indentation

```
if 5 > 2:  
    print("Five is greater than two!")
```

Where in other programming languages the indentation in code is for readability only, the indentation in Python is very important.

Python uses indentation to indicate a block of code.

Python will give you an error if you skip the indentation:

Example

Syntax Error:

```
if 5 > 2:  
print("Five is greater than two!")
```

ERROR

The number of spaces is up to you as a programmer, the most common use is four, but it has to be at least one.

```
if 5 > 2:  
    print("Five is greater than two!")  
if 5 > 2:  
    print("Five is greater than two!")
```

Acceptable

You have to use the same number of spaces in the same block of code, otherwise Python will give you an error:

Example

Syntax Error:

```
if 5 > 2:
    print("Five is greater than two!")
    print("Five is greater than two!")
```

ERROR

Variables in Python:

```
x = 5
y = "Hello, World!"
```

Text Type: `str`

Numeric Types: `int`, `float`, `complex`

Sequence Types: `list`, `tuple`, `range`

Mapping Type: `dict`

Set Types: `set`, `frozenset`

Boolean Type: `bool`

Binary Types: `bytes`, `bytearray`, `memoryview`

None Type: `NoneType`

```
thislist = ["apple", "banana", "cherry"]  
print(thislist)
```

Lists

Since lists are indexed, lists can have items with the same value:

Example

Lists allow duplicate values:

```
thislist = ["apple", "banana", "cherry", "apple", "cherry"]  
print(thislist)
```

Print the second item of the list:

```
thislist = ["apple", "banana", "cherry"]  
print(thislist[1])
```

Print the last item of the list:

```
thislist = ["apple", "banana", "cherry"]  
print(thislist[-1])
```

Return the third, fourth, and fifth item:

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]  
print(thislist[2:5])
```

Tuple

Tuples are used to store multiple items in a single variable.

Tuple is one of 4 built-in data types in Python used to store collections of data, the other 3 are List, Set, and Dictionary, all with different qualities and usage.

A tuple is a collection which is ordered and **unchangeable**.

Tuples are written with round brackets.

Create a Tuple:

```
thistuple = ("apple", "banana", "cherry")  
print(thistuple)
```

Unchangeable

Tuples are unchangeable, meaning that we cannot change, add or remove items after the tuple has been created.

Set

Sets are used to store multiple items in a single variable.

Set is one of 4 built-in data types in Python used to store collections of data, the other 3 are List, Tuple, and Dictionary, all with different qualities and usage.

A set is a collection which is *unordered*, *unchangeable**, and *unindexed*.

Create a Set:

```
thisset = {"apple", "banana", "cherry"}  
print(thisset)
```

Create a class named MyClass, with a property named x:

```
class MyClass:  
    x = 5
```

Create an object named p1, and print the value of x:

```
p1 = MyClass()  
print(p1.x)
```

The `__init__()` Function

The examples above are classes and objects in their simplest form, and are not really useful in real life applications.

To understand the meaning of classes we have to understand the built-in `__init__()` function.

All classes have a function called `__init__()`, which is always executed when the class is being initiated.

Use the `__init__()` function to assign values to object properties, or other operations that are necessary to do when the object is being created:

Create a class named Person, use the `__init__()` function to assign values for name and age:

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
p1 = Person("John", 36)
```

```
print(p1.name)
```

```
print(p1.age)
```

```
class Point:
    def __init__(self, x = 0, y = 0):
        self.x = x
        self.y = y
```

```
    def add_points(self, other):
        x = self.x + other.x
        y = self.y + other.y
        return Point(x, y)
```

```
p1 = Point(1, 2)
```

```
p2 = Point(2, 3)
```

```
p3 = p1.add_points(p2)
```

```
print((p3.x, p3.y))    # Output: (3, 5)
```

```
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    # overload < operator
    def __lt__(self, other):
        return self.age < other.age

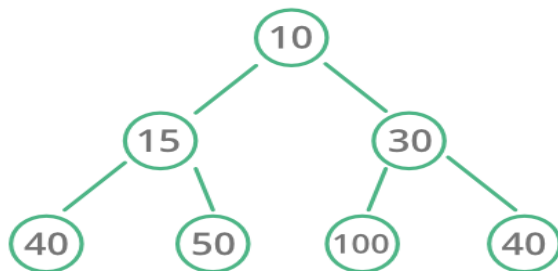
p1 = Person("Alice", 20)
p2 = Person("Bob", 30)

print(p1 < p2) # prints True
print(p2 < p1) # prints False
```

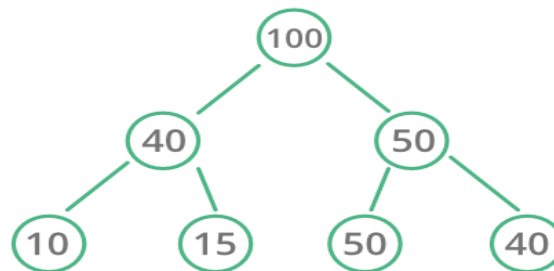
A heap queue or priority queue is a data structure that allows us to quickly access the smallest (min-heap) or largest (max-heap) element. A heap is typically implemented as a binary tree, where each parent node's value is smaller (for a min-heap) or larger (for a max-heap) than its children. However, in Python, heaps are usually implemented as min-heaps which means the smallest element is always at the root of the tree, making it easy to access.

heapq module allows us to treat a [list](#) as a heap, providing efficient methods for adding and removing elements.

Heap Data Structure



Min Heap



Max Heap

```
h = [10, 20, 15, 30, 40]
```

```
heapq.heapify(h)
```

```
# Appending an element
```

```
heapq.heappush(h, 5)
```

```
# Pop the smallest element from the heap
```

```
min = heapq.heappop(h)
```

```
print(h)
```

```
print("Smallest:", min)
```

```
print(h)
```

Output

```
[10, 20, 15, 30, 40]
```

```
Smallest: 5
```

```
[10, 20, 15, 30, 40]
```

.....

For 2-dimensional vectors:

$$\text{Manhattan distance} = |x_1 - x_2| + |y_1 - y_2|$$

For 3-dimensional vectors:

$$\text{Manhattan distance} = |x_1 - x_2| + |y_1 - y_2| + |z_1 - z_2|$$

General formula for n-dimensional vectors:

$$\text{Manhattan distance} = |x_1 - x_2| + |y_1 - y_2| + \dots + |v_{n1} - v_{n2}|$$

Scalars, Vectors, Matrices & Tensors

1

$\begin{bmatrix} 1 \\ 2 \end{bmatrix}$

$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

$\begin{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} & \begin{bmatrix} 3 & 2 \end{bmatrix} \\ \begin{bmatrix} 1 & 7 \end{bmatrix} & \begin{bmatrix} 5 & 4 \end{bmatrix} \end{bmatrix}$

Scalar

Vector

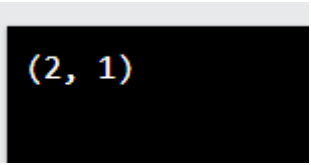
Matrix

Tensor

Heapq internally uses less than operator

```
x = divmod(5, 2)
```

```
print(x)
```



```
(2, 1)
```

8 puzzle problem solve

1	2	3
4	5	6
0	7	8

1	2	3
4	5	6
7	8	0

Example usage

`initial = [1, 2, 3, 4, 5, 6, 0, 7, 8]`

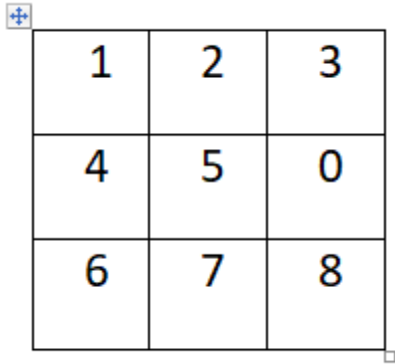
`goal = [1, 2, 3, 4, 5, 6, 7, 8, 0]`

1	2	3
0	4	5
6	7	8

1	2	0
3	4	5
6	7	8

```
def possible_moves(board):  
    moves = []  
    empty_index = board.index(0)  
    if empty_index % 3 > 0: # Move left  
        moves.append(-1)  
    if empty_index % 3 < 2: # Move right  
        moves.append(1)  
    if empty_index > 2: # Move up  
        moves.append(-3)  
    if empty_index < 6: # Move down  
        moves.append(3)  
    return moves
```

```
def manhattan_distance(board, goal):  
    distance = 0  
    for i in range(1, 9):  
        x1, y1 = divmod(board.index(i), 3)  
        x2, y2 = divmod(goal.index(i), 3)  
        distance += abs(x1 - x2) + abs(y1 - y2)  
    return distance
```



1	2	3
4	5	0
6	7	8

```
def apply_move(board, move):  
    new_board = board[:]  
    empty_index = new_board.index(0)  
    new_index = empty_index + move  
    new_board[empty_index], new_board[new_index] = new_board[new_index], new_board[empty_index]  
    return new_board
```

```
def a_star_search(initial_state, goal_state):
    explored = set()
    priority_queue = []
    initial = PuzzleState(initial_state, None, None, 0, manhattan_distance(initial_state, goal_state))
    heapq.heappush(priority_queue, initial)

    while priority_queue:
        current_state = heapq.heappop(priority_queue)
        explored.add(tuple(current_state.board))

        if current_state.board == goal_state:
            return current_state

        for move in possible_moves(current_state.board):
            new_board = apply_move(current_state.board, move)
            if tuple(new_board) not in explored:
                new_state = PuzzleState(new_board, current_state, move, current_state.depth + 1, current_state.depth + 1 + manhattan_distance(new_bo
                heapq.heappush(priority_queue, new_state)

    return None
```

1	2	3
4	5	0
6	7	8

```
def possible_moves(board):  
    moves = []  
    empty_index = board.index(0)  
    if empty_index % 3 > 0: # Move left  
        moves.append(-1)  
    if empty_index % 3 < 2: # Move right  
        moves.append(1)  
    if empty_index > 2: # Move up  
        moves.append(-3)  
    if empty_index < 6: # Move down  
        moves.append(3)  
    return moves
```